ACADEMIC & INDUSTRY ARCHITECTURE SPECIFICATION • REAL-DATA BLOCKCHAIN

Blockchain-Based Digital Document Locker (Eman)

A decentralized, tamper-evident, sovereign digital document storage, verification, and sharing architecture designed as a zero-trust alternative to centralized repositories.

AUTHOR / ROLL NO.
Shah Emanul Haque
(25BCE1463)

ARCHITECTURE
5-Tier Decentralized
(EVM + IPFS + Web3)

SMART CONTRACTS
Solidity 0.8.24
Registry & AccessControl

STORAGE TOPOLOGY
On-Chain Hashes & CIDs
Off-Chain AES-256 IPFS

01 Project Overview & Problem Context

Traditional government and enterprise document repositories (such as the centralized **DigiLocker** platform) store citizens' high-value credentials—including educational degrees, identity records, property titles, and health certificates—on centralized servers managed by cloud providers and institutional administrators. While this has advanced paperless workflows, centralized architectures suffer from critical systematic flaws:

Vulnerability Dimension	Traditional Centralized Locker (DigiLocker)	Eman Blockchain Document Locker
Architecture & Single Point of Failure	Centralized database and monolithic cloud servers. If the central provider experiences an outage, data corruption, or DDOS attack, nationwide verification halts. Over 3.68 crore accounts have historically been at risk in centralized breaches.	**Decentralized & Multi-Node:** Ethereum EVM consensus nodes coupled with distributed InterPlanetary File System (IPFS) peer-to-peer storage. No single point of failure can disable the network or compromise data availability.
Data Sovereignty & Custody	Platform administrators hold direct database access and can view, edit, or delete records without user approval. The citizen is a passive beneficiary rather than a sovereign owner.	**Self-Sovereign Ownership:** Controlled entirely by the user's private key (via MetaMask / EIP-1193). Smart contracts forbid any third party or administrator from revoking, altering, or sharing documents without owner signatures.
Tamper Detection & Verification Speed	Database records can be modified by rogue insiders or malicious SQL injections. Verification requires manual, out-of-band communication with issuing bodies taking hours or days.	**Deterministic SHA-256 Anchors:** Documents are fingerprinted client-side before upload. The digest is stored permanently on-chain. Immediate tamper detection in under 2 seconds.
Confidentiality at Rest	Files are stored plaintext or with server-side managed encryption keys. Cloud providers, server admins, and law enforcement can read documents directly off disk.	**Client-Side Authenticated Encryption:** Symmetrically encrypted in the browser with AES-256-GCM prior to transmission. Storage nodes and backend handle only high-entropy ciphertext.

Guiding Principle: Absolute Non-Synthetic Data Policy

The Eman Digital Document Locker is engineered as a real-world Web3 production system. Every cryptographic key, wallet signature, SHA-256 digest, IPFS Content Identifier (CID), transaction receipt, block number, and audit log originates from genuine client operations or live EVM execution. No synthetic, fake, or mock fallback values exist in the application.

02 System Architecture & Layered Topology

The system is designed as an institutional-grade, zero-trust **5-Tier Decentralized Architecture**. Responsibilities are strictly segregated to isolate private cryptographic keys from public ledgers and backend relays, guaranteeing that raw plaintext documents never touch untrusted infrastructure.

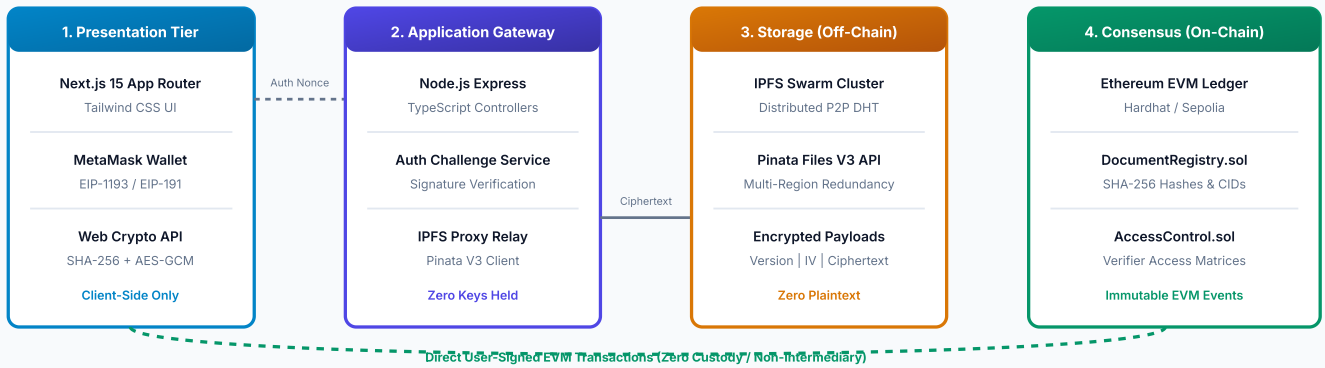


Figure 1: Complete 5-Tier Architecture Model Showing Separation of Storage, Key Custody, and Consensus

Detailed Tier Architecture & Responsibilities

1. Presentation Tier (Next.js 15 & MetaMask)

Runs directly within the user's browser. Responsible for local file selection, computing deterministic SHA-256 hashes, generating ephemeral 256-bit AES-GCM keys via the W3C Web Crypto API, packing serialized binary envelopes, and interfacing with MetaMask (via `ethers.js v6`) for EIP-1193 transaction signing.

2. Application Gateway (Node.js & Express)

A stateless REST API layer. Implements cryptographic wallet login using EIP-191 personal sign nonces (eliminating username/password vulnerabilities). Relays encrypted payloads to Pinata IPFS nodes, extracts CIDs, and filters indexed EVM event logs for the frontend audit timeline.

3. Consensus Layer (Ethereum EVM)

Composed of Solidity 0.8.24 smart contracts (`DocumentRegistry.sol` and `AccessControl.sol`). Acts as the single, tamper-proof source of truth for document ownership, file integrity hashes, IPFS storage pointers, verifier permissions, and permanent revocation flags.

4. Decentralized Storage (IPFS & Pinata V3)

A content-addressed peer-to-peer distributed storage network. Stores only high-entropy, encrypted binary blobs. Documents remain mathematically unreadable on the public network without the corresponding 256-bit AES symmetric key held exclusively by the owner.

03 The Registry Module: DocumentRegistry.sol

The **DocumentRegistry** contract is the authoritative on-chain foundation of the platform. It provides permanent proof of existence, integrity anchors, decentralized storage references, and sovereign revocation controls.

3.1 Storage Layout & Data Structures

The contract optimizes Ethereum storage slots by maintaining an unalterable struct and two primary $O(1)$ mapping structures:

```
// Immutable document record anchored in EVM consensus storage
struct Document {
    bytes32 documentId;   // 32-byte cryptographically secure opaque unique ID
    address owner;        // Sovereign wallet address (msg.sender at registration)
    bytes32 documentHash; // Deterministic SHA-256 digest of original plaintext bytes
    string  ipfsCid;       // Content Identifier (CID v0 / v1) of encrypted payload
    uint256 registeredAt; // EVM block timestamp of registration
    bool    revoked;      // Permanent sovereign revocation status flag
}

// Primary index: O(1) direct document retrieval by documentId
mapping(bytes32 ⇒ Document) private _documents;

// Secondary index: Enumeration of all document IDs owned by a specific address
mapping(address ⇒ bytes32[]) private _userDocuments;
```

3.2 Opaque Cryptographic 32-Byte Identifier Strategy

Unlike naive blockchain registries that use auto-incrementing numbers (e.g. 1, 2, 3) or plaintext filenames (e.g. "degree.pdf"), the Eman locker enforces an **opaque 32-byte cryptographic identifier**:

- **Generation:** Created client-side via 256 bits of CSPRNG entropy: `ethers.hexlify(ethers.randomBytes(32))`.
- **Information Hiding:** Prevents attackers from scanning or guessing document IDs, inferring user portfolio sizes, or intercepting document classifications from sequential sequences.
- **Collision Prevention:** The contract requires `_documents[documentId].owner == address(0)`. If a duplicate ID is submitted, it immediately reverts with custom error `DocumentAlreadyExists(documentId)`.

3.3 Core Operations & State Transitions

Method Signature	Visibility	Operational Semantics & Security Invariants
`registerDocument(bytes32, bytes32, string)`	External (Tx)	Validates non-zero ID, non-zero hash, and non-empty CID. Sets `doc.owner = msg.sender`. Sets `registeredAt = block.timestamp` and `revoked = false`. Emits `DocumentRegistered`.
`getDocument(bytes32)`	External View	Returns complete `Document` struct. Reverts with `DocumentNotFound` if unallocated. Used for verification and audit.
`getUserDocuments(address)`	External View	Returns dynamic array of 32-byte document IDs registered by the queried wallet address. Powers the user's dashboard document catalog.
`revokeDocument(bytes32)`	External (Tx)	Restricted strictly to `msg.sender == doc.owner`. Sets `revoked = true`. Reverts if already revoked (`DocumentAlreadyRevoked`). Emits `DocumentRevoked`.
`isRevoked(bytes32)` / `documentExists(bytes32)`	External View	Gas-optimized helper view queries invoked by downstream contracts (`AccessControl.sol`) and backend verification engines.

> **Permanent Revocation Semantics vs. Data Deletion**
> Because blockchain transaction logs cannot be erased, revocation does not attempt to purge historical blocks. Instead, it marks the document as permanently invalid in current EVM state. Subsequent verifications detect `revoked == true` and instantly warn third parties that the credential has been formally invalidated by its owner.

04 Smart Contracts Used: Architecture & AccessControl.sol

The blockchain layer employs a modular, decoupled architecture consisting of two primary smart contracts written in **Solidity 0.8.24**: `DocumentRegistry.sol` and `AccessControl.sol`, linked via `IDocumentRegistry.sol`.

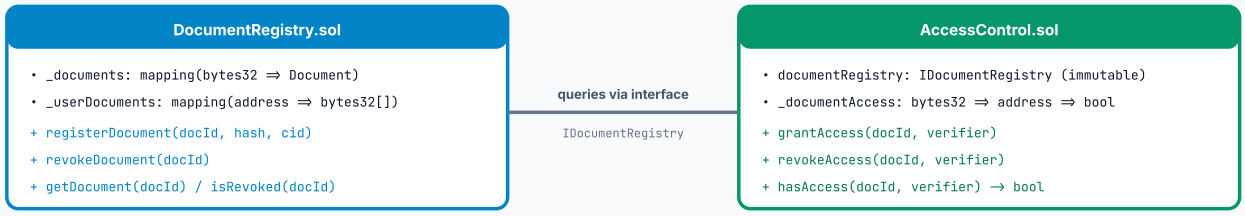


Figure 2: Decoupled Smart Contract Topology — AccessControl Interfaces with DocumentRegistry

4.1 AccessControl.sol Architecture

The `AccessControl` contract manages on-chain authorization for third-party verifiers, universities, banks, and employer wallets. It guarantees that only the authoritative document owner recorded in `DocumentRegistry` can grant or revoke access.

Key Technical Invariants:

- **Immutable Registry Pointer:** The contract takes the deployed address of `DocumentRegistry` upon construction and binds it to an `immutable` variable (`public immutable documentRegistry`), permanently neutralizing address spoofing or contract redirection exploits.
- **Sparse Mapping Model:** Avoids unbounded dynamic arrays by storing grants in a nested mapping: `mapping(bytes32 ⇒ mapping(address ⇒ bool)) private _documentAccess;` . Historical verifier sets are indexed through EVM events.
- **Composite `hasAccess` Logic:** Computes effective access by synthesizing registry state and direct permissions:

```
function hasAccess(bytes32 docId, address verifier) external view returns (bool) {
    if (docId == bytes32(0) || verifier == address(0)) return false;
    if (!documentRegistry.documentExists(docId)) return false;
    if (documentRegistry.isRevoked(docId)) return false;
    return _documentAccess[docId][verifier];
}
```

  Crucial Security Benefit: If a document is revoked on `DocumentRegistry` , `hasAccess` instantly drops to `false` for every verifier simultaneously without requiring separate transactions!

4.2 Smart Contract Security & Engineering Standards

Engineering Standard	Implementation Detail
Solidity Custom Errors	Replaces costly error strings with typed custom errors (e.g. `error NotDocumentOwner(bytes32, address)`). Saves ~2,000 gas per revert and minimizes deployed bytecode footprint.
Zero Administrative Master Keys	No contract owner, multi-sig admin, or governance role can override document ownership, view private metadata, or manipulate permissions.
Checks-Effects-Interactions (CEI)	All input checks and state mutations occur prior to event emissions. Neither contract accepts ether or invokes external untrusted addresses, eliminating reentrancy attack surfaces.
Indexed Event Logging	Emits indexed events (`DocumentRegistered` , `DocumentRevoked` , `AccessGranted` , `AccessRevoked`) with `bytes32 indexed documentId` and `address indexed owner` , enabling zero-gas audit trail reconstruction via standard JSON-RPC queries.

05 Storage Architecture: On-Chain vs. Off-Chain

In decentralized engineering, naive storage models that place large files directly on-chain encounter the **Blockchain Storage Trilemma**: exorbitantly high gas costs (several thousand dollars per megabyte), network bloat, and total loss of privacy on public ledgers.

The Eman system solves this through a rigorous **Hybrid Storage Topology**: anchoring cryptographic integrity on-chain while storing client-encrypted ciphertext on decentralized peer-to-peer storage.

Storage Dimension	On-Chain Storage (Ethereum EVM)	Off-Chain Storage (IPFS / Pinata V3)
Physical Medium	Ethereum distributed consensus ledger (immutable block storage, SSTORE slots).	InterPlanetary File System (IPFS) peer-to-peer libp2p DHT, pinned via Pinata V3 cluster.
Stored Artifacts	• `bytes32 documentId` (Unique identifier) • `bytes32 documentHash` (SHA-256 fingerprint) • `string ipfsCid` (IPFS content locator) • `address owner` & block timestamp • Access grant booleans	• **AES-256-GCM Encrypted Payloads** • Self-contained binary envelope [Version | IV | Ciphertext + Tag] • Zero plaintext, zero unencrypted headers, zero personal metadata.
Cost Profile	$O(1)$ constant storage cost (~3 storage slots = ~60,000 gas per document). Independent of physical file size.	Scales linearly with bandwidth and storage; handles documents from 1 KB up to 100+ MB with zero blockchain gas penalty.
Confidentiality Model	Public state. Accessible to anyone on-chain, but stores **zero** sensitive content or encryption keys.	Distributed public swarm. **Mathematically unbreakable** without the client's 256-bit symmetric AES key.

5.1 Client-Side AES-256-GCM Authenticated Encryption

Files are encrypted directly within the user's browser memory using **AES-256 in Galois/Counter Mode (GCM)** (NIST SP 800-38D) via the browser's native Web Crypto API. Key cryptographic properties include:

- **256-Bit Cryptographic Entropy:** Every document receives an independently generated symmetric key created via `crypto.subtle.generateKey`.
- **96-Bit Nonce Freshness:** A unique 96-bit Initialization Vector (IV) is generated per encryption operation via `crypto.getRandomValues(new Uint8Array(12))`. IV reuse is cryptographically prevented.
- **128-Bit GHASH Authentication Tag:** AES-GCM generates a 16-byte Poly1305/GHASH tag. Decryption strictly validates this tag; any single-bit alteration in the ciphertext or IV causes decryption to immediately fail closed with `AuthenticationError`.

5.2 Canonical Binary Payload Serialization Envelope

To prevent metadata leakage and ensure uniform persistence on IPFS nodes, the encrypted output is packed into a canonical binary format:



Figure 3: Canonical Binary Payload Format Persisted to IPFS Nodes

5.3 Strict Key Isolation & Custody Rules

The raw 256-bit AES symmetric key is held strictly in temporary client browser memory during document creation. It is **never** transmitted to smart contracts, never saved in browser storage or cookies, and never sent to the backend API. The owner downloads the key file locally (`.key` format) to retain sovereign custody over decryption.

06 Verification Workflow & Multi-Tier Tamper Detection

Verification is the ultimate operational test of the document locker. The system supports two distinct, cryptographically rigorous verification workflows:

Workflow A: Candidate File Verification

An operator or verifier holds a candidate document file (e.g. PDF degree certificate) and wishes to confirm its authenticity. The client hashes the candidate bytes locally using SHA-256 and submits the hash to the backend. The system compares it against `DocumentRegistry.documentHash` and checks revocation state. Verdict: **VERIFIED** in < 2 seconds.

Workflow B: Stored Object Audit

An authorized user fetches the encrypted payload from IPFS via its CID, loads the owner's `.key` file, decrypts the payload client-side, and recomputes the SHA-256 hash over the recovered bytes, confirming byte-for-byte fidelity with the blockchain record.

6.1 Multi-Tier Attack Vectors & Defense Outcomes

Threat / Attack Vector	Target Layer	Cryptographic Detection Mechanism	System Outcome
1-Byte Document Alteration	Candidate Document	Avalanche effect of SHA-256 alters digest completely; fails on-chain comparison.	TAMPER_DETECTED
File Substitution	Candidate Document	Candidate hash does not match canonical `documentHash` in `DocumentRegistry`.	TAMPER_DETECTED
Ciphertext Tampering on IPFS	IPFS Encrypted Blob	AES-256-GCM GHASH authentication tag verification fails immediately during Web Crypto decryption.	DECRYPTION_FAILED
IV or Tag Manipulation	Binary Payload Envelope	Altering 12-byte IV or 16-byte tag causes Galois field authentication check to throw exception.	DECRYPTION_FAILED
Revocation by Owner	Smart Contract State	`DocumentRegistry.isRevoked()` returns true; `hasAccess()` drops to false.	REVOKED
Unauthorized Verifier	AccessControl.sol	Caller address lacks affirmative mapping in `_documentAccess`; HTTP 403 Forbidden.	UNAUTHORIZED

6.2 Technology Stack Summary

Subsystem	Technologies & Standards	Operational Role
Frontend	Next.js 15, React 19, TypeScript, Tailwind, ethers.js v6	Sovereign Web3 UI, client-side encryption, MetaMask signer integration.
Cryptography	W3C Web Crypto API, AES-256-GCM, SHA-256 (FIPS 180-4)	In-memory envelope encryption, CSPRNG key generation, file fingerprinting.
Backend API	Node.js 22, Express.js, TypeScript, EIP-191 Auth	Stateless challenge nonce issuer, Pinata V3 IPFS relay, event log aggregator.
Blockchain	Solidity 0.8.24, Hardhat, TypeChain, Ethereum EVM	Consensus-backed document registry and verifier access control contracts.
Decentralized Storage	IPFS, Pinata Files V3 REST API	Content-addressed distributed storage of encrypted binary payloads.

Conclusion & Project Synthesis

The Eman Blockchain-Based Digital Document Locker achieves a complete synthesis of decentralized sovereign ownership, cryptographic confidentiality, and consensus-driven tamper evidence. By maintaining minimal, immutable integrity anchors on-chain (SHA-256 hashes and IPFS CIDs) and storing client-side encrypted payloads off-chain on IPFS, the system eliminates centralized single points of failure while providing instantaneous, trustless credential verification.